Smaller clause-discharge checkers for iPRA

A measured implementation reduction and a deterministic equality rule

Research prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

We give two complete presentations of full first-order intuitionistic primitive recursive arithmetic over the same normal scalar iterator syntax. The implementation recommended when checker size is decisive retains the existing three-rule witness-discharge calculus and reduces its complete normalized Python checker from 2,273 to 2,010 bytes. The second presentation has a new deterministic discharge rule: the checker reads an equality from the opened witness clause at the fixed argument 1, so the certificate supplies neither an equality proof nor a substitution target. It uses the same three inference families and the same single reflexivity axiom clause. Its complete checker has 2,058 bytes, 653 significant tokens and 729 AST nodes. The smaller explicit checker has 636 tokens and 708 AST nodes.

We prove exact equivalence on arbitrary well-scoped clause sequents. The noncircular completeness argument first derives numerical cases without equality transport, and then uses an inherited two-row selector package with the equality row moved to the fixed inspection position. Both presentations inherit whole-domain bi-interpretability and equiconsistency with iPRA, actual equality, and identity numerical composite maps.

The deterministic rule costs 48 bytes more than the matched explicit rule; its benefit is a smaller primitive certificate interface. The implementation reductions and the logical reduction are therefore reported separately. Neither checker beats the repository's distinct 516-byte compact codec or 788-byte validated public-goal codec, and no absolute minimum is claimed. The package contains the complete source, mathematical foundations, executable proof translations, adversarial checks, and a bounded-certificate variant. New proofs are written mathematics, not a new Lean formalization.

Contents

1 Result and the comparison that the measurements support **3**

2 Equality read from one fixed row **4**

3 The full iPRA comparison and both interpretation composites **9**

4 Certificate syntax and the complete trusted program **11**

5 Normalization, bounded checking, and the consistency claim **13**

6 Verification and provenance **14**

A Computation normalization and the raw comparison **14**

Attribution and role of the following foundations **17**

B Complementary introductions with one local assumption **17**

C Arithmetic strength and both ordinary interpretation composites **22**

D Internal scalar predecessor bootstrap **24**

E From literal-equation induction to full intuitionistic PRA **28**

F Pairing, primitive recursion, and internal numerical composites **34**

G Complete standalone acceptance programs **36**

H Reproduction and file map **38**

1 Result and the comparison that the measurements support

The request is to simplify the current proposal at `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. We inspected the repository at the fixed revision `ffa94f76bc01`, identified in full in the bibliography, including the six archives in `docs/incoming/`. That snapshot matters: the incoming witness-discharge proposal already has three inference families, although the main README's most compact clause interface has four families. The closest logical predecessor is thus the incoming witness-discharge package [2], not an earlier eight-rule native presentation.

Two distinct simplifications result. First, a complete implementation can infer the next fresh numerical parameter from its clause context, use one shared traversal for shifting and opening, and expose a minimal two-field standalone input. This preserves the predecessor's calculus and lowers all three primary source coordinates. Second, the equality proof in witness discharge can be replaced by a fixed, syntactically recognizable consequence of the opened clause. This lowers the rule's proof-premise count from three to two and removes a term parameter, with no loss of arithmetic or logic. Its selector must be counted, however, and it makes the matched program slightly longer.

1.1 Which delivered presentation should be used?

If the objective is the size of a straightforward complete proof checker, use `code/checker_explicit.py`. It is the smallest measured member of the delivered pair. If the objective is to minimize the equality information that a proof producer supplies, use `code/checker_deterministic.py`. The latter is the new mathematical calculus. Both have the same numerical and formula language, the same ordinary meaning, and the same consequences under every canonical clause context. The explicit implementation is included as a full deliverable, not merely as a control used to make a comparison favorable.

Complete standalone module	Bytes	Tokens	AST	Gzip-9
Prior witness discharge	2273	732	813	903
Prior uniform binding	2238	693	764	866
Delivered explicit discharge	2010	636	708	791
Delivered deterministic discharge	2058	653	729	805

These counts include the parser invocation, typing, scopes, normalizer, capture-avoiding substitution, every inference, the complete input protocol, and final acceptance. They are reproduced by the supplied measurement script on CPython 3.12.14: remove documentation strings, apply `ast.unparse`, append one LF, count UTF-8 bytes, significant lexical tokens, and all nodes visited by `ast.walk`. Gzip uses level 9 and timestamp zero and is informational, not an objective being minimized. The exact raw-source sizes and hashes are also recorded.

The explicit checker saves 263 normalized bytes (11.57%), 96 tokens (13.11%), and 105 AST nodes (12.92%) against its direct predecessor. The deterministic checker saves 215 bytes (9.46%), 79 tokens (10.79%), and 84 nodes (10.33%) against that predecessor. In the matched delivered pair, automatic extraction adds 48 bytes, 17 tokens, 21 nodes and 14 gzip bytes. Thus the theorem about the new rule is not an explanation of all the implementation savings.

1.2 Scope of the minimum claim

The inspected readable/transparent incoming checkers have coordinate minima 2,238 bytes, 666 tokens and 764 AST nodes, achieved by different programs [3, 4]. Both delivered programs are below those three numbers. This is a scoped comparison, not a lower-bound theorem. The repository also contains heavily fused compact programs, including a 516-byte checker whose public-goal grammar is a precondition and a 788-byte native public-goal validator. Those are smaller than this delivery. There is no justified claim here to have found the globally simplest axiomatization, or even the shortest checker across all codecs in the repository. The positive result is a genuine logical restriction together with smaller complete implementations in the explicitly compared family.

1.3 What is inherited and what is new

Inherited components are the scalar iterator arithmetic, the paired D binder, the witness-dependent introduction, optional-premise clauses, the fixed reflexivity clause, full computation normalization, the two-row selector construction, and the arithmetic bridge to iPRA. The new logical step is to designate row 1 in the elimination rule and read the equality from it automatically, replacing the explicit equality subproof and target term. The rows of the inherited selector package are interchanged to match that fixed position. The new implementation work concerns the context-derived scope, common traversal, and standalone protocol. All of these costs remain in the measured modules.

The following sections state the complete calculus and prove the new step. The arithmetic and logical foundations are reproduced with attribution in the appendices. They are derived comparisons and do not enlarge the list of primitive rules.

2 Equality read from one fixed row

The new reduction is local to witness discharge. It removes the equality proof that the preceding *Name* rule asks the certificate to supply. Instead, the rule reads one specified instance of its own opened clause. The zero instance must be the fixed reflexive equation $0 = 0$, and the successor instance must name the fresh witness. These are decidable syntactic tests, not proof-search operations.

2.1 Setting and complete discharge rule

Write N for full computation normalization of scalar iterator terms and of the numerical terms in formulas. The reductions are

$$I(0, a; u.h) \longrightarrow a, \qquad I(Sn, a; u.h) \longrightarrow h[I(n, a; u.h)/u].$$

Terms and formulas in inference labels are normal forms. Substitution is capture avoiding and followed by N. In particular, the argument uses the inherited normalization–substitution identity

$$N\big(N(F)[t/b]\big) = N\big(F[t/b]\big). \tag{1}$$

This is a metasyntactic identity. It is not an application of an object-language equality rule. No numerical variable ranges over programs or function descriptions.

Before normalization the complete numerical grammar is

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h),$$

where only u is bound in h, and the other arguments do not bind. The native formulas are equations and $D_{x,y}B$. Its ordinary reading is

$$\big(D_{x,y}B\big)^{\circ} = \exists x\big(B^{\circ}(x,0) \to \forall y\, B^{\circ}(x,y)\big).$$

A context consists of reusable clauses $[\Pi; y.A(y)]$, with zero or one antecedent in Π; the displayed y is not free in the antecedent. An ordinary assumption A is the unconditional constant clause $[\,; v.A]$, where v is unused. Every derivation has the fixed initial clause

$$\mathcal{R} = [\,; y.y = y].$$

Its instances supply reflexivity by the ordinary clause-use instruction. The designated contradiction is still $0 = S0$.

The Use instruction is unchanged: from the antecedent of a selected clause, if present, infer its consequent at any supplied numerical term. The Intro instruction is also unchanged. At witness t and fresh z, it derives $D_{x,y}B$ from a branch proving $N(B(t, Sz))$. The branch has the ordinary assumption $N(B(t,z))$ precisely when B is an equation and t is literally 0; otherwise it has $N(B(t,0))$. The witness and the matrix's passive parameters are outer scoped, and z is fresh for all retained data.

For the new discharge instruction, first prove $D_{x,y}B$ and open it at a fresh numerical parameter b. Write

$$\mathcal{C}_b = [N(B(b,0)); y.N(B(b,y))], \qquad 1 = S0.$$

Define the term $\tau_B(b)$ deterministically:

$$\tau_B(b) = \begin{cases} t, & N(B(b,0)) \text{ is literally } 0 = 0 \text{ and } N(B(b,1)) \text{ is literally } b = t, \\ b, & \text{otherwise.} \end{cases} \tag{2}$$

There is no certificate for either test. The fresh witness must be the *entire left term* of the tested equation, not merely one of its subterms. The rule is

$$\frac{\Gamma \vdash D_{x,y}B \qquad \Gamma, \mathcal{C}_b \vdash F}{\Gamma \vdash N(F[\tau_B(b)/b])}, \qquad b \notin \mathrm{FV}\big(N(F[\tau_B(b)/b])\big). \tag{Row}$$

The parameter b is fresh for the retained context and the matrix's passive parameters. Replacement is simultaneous and performed once: inserted copies of $\tau_B(b)$ are not revisited. The selected target may itself contain b. The condition required for discharge is absence of b from the *final normalized result*, not absence from that target.

Use, Intro, and Row, together with $\mathcal{R}$ and the specified canonical syntax, are the entire presentation. Full normalization is part of its trusted specification and implementation. There are no additional primitive equality, computation, successor, or induction rules. This still counts as three instruction families; the reduction is from three proof premises to two in the discharge instruction and the deletion of its independently supplied equality data.

2.2 Soundness and the equality-free preliminaries

Let $\mathcal{K}_=$ denote the preceding full-normal Name calculus, whose discharge is the explicit rule

$$\frac{\Gamma \vdash D_{x,y}B \qquad \Gamma, \mathcal{C}_b \vdash b = t \qquad \Gamma, \mathcal{C}_b \vdash F}{\Gamma \vdash N(F[t/b])}, \quad b \notin \mathrm{FV}(N(F[t/b])). \tag{Name}$$

Here b has the same freshness requirement as in Row, and t is well scoped in the local context; it may contain b. Let $\mathcal{K}_{\mathrm{row}}$ denote the new calculus.

Lemma 2.1 (The rule supplies its own equality)**.** *In the opened context $\Gamma, \mathcal{C}_b$, one can derive $b = \tau_B(b)$ using only Use and $\mathcal{R}$.*

Proof. If the test in (2) succeeds, reflexivity gives its zero antecedent $0 = 0$, and Use of $\mathcal{C}_b$ at 1 gives exactly $b = t$. If the test fails, Use of $\mathcal{R}$ at b gives $b = b$. □

Corollary 2.2 (Embedding into the preceding calculus)**.** *Every Row inference is derivable in $\mathcal{K}_=$ with the same end sequent. Hence every new derivation is sound for the preceding ordinary interpretation.*

Proof. Retain the major and formula branches, insert the equality proof of Lemma 2.1, and use the old Name inference. Its result and eigenparameter condition are exactly those of Row. □

The same argument works directly in ordinary intuitionistic equality logic: equality gives $F[\tau_B(b)/b]$, normalization preserves its meaning, and the result's b-freedom permits existential elimination. This also explains why no occurs check on the selected target is needed.

Lemma 2.3 (Ordinary paired elimination)**.** *From $\Gamma \vdash D_{x,y}B$ and $\Gamma, \mathcal{C}_b \vdash A$, with b absent from A, one can derive $\Gamma \vdash A$.*

Proof. Use Row with the given formula branch A. Regardless of which target the test selects, $N(A[\tau_B(b)/b])$ is literally A: there is no b to replace, and A is already normal. In particular, this argument does not assume that the rule takes its fallback branch. □

Weakening, injective fresh renaming, and cut of an ordinary assumption are admissible by direct induction on derivations. Freshen local parameters before insertion. An ordinary-assumption cut replaces uses of its unconditional constant clause by the supplied proof. The matrix and deterministic test at a Row node are unchanged by such insertion. Injective renaming commutes with normalization and preserves and reflects the tests against $0 = 0$ and against the distinguished fresh parameter. These structural arguments do not assume numerical substitution of derivations or any equality-congruence theorem.

For a variable w unused in A, put $q_y A(y) = D_{w,y}A(y)$.

Lemma 2.4 (Numerical cases before equality transport)**.** *The following rules are derivable using only Use, Intro, Lemma 2.3, and $\mathcal{R}$:*

$$\frac{\Gamma, A(0) \vdash A(Sz)}{\Gamma \vdash q_y A(y)},$$

$$\frac{\Gamma \vdash q_y A(y) \qquad \Gamma \vdash A(0)}{\Gamma \vdash A(t)},$$

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}.$$

Here all instances are normalized, and the successor-branch parameter is fresh for the retained context and the template's passive parameters.

Proof. For the first rule, introduce the unused witness w at 1. This forces Intro's case mode even when A is an equation, because the chosen witness is not literally 0. Its branch is precisely the supplied $A(0) \vdash A(Sz)$.

For the second rule, open $q_y A$ by ordinary paired elimination. Since its witness coordinate is unused, the opened clause is $[A(0); y.A(y)]$. Use it at t with the given proof of $A(0)$, then discharge the unused witness. For the third rule, weaken its successor proof by the unused assumption $A(0)$ and combine the first two rules. □

This is numerical case distinction, not equation induction: its successor branch has no induction hypothesis. It has been obtained before, and without, any transport principle.

2.3 Recovering arbitrary equality from two numerical rows

Define a normalizing selector and a short parity term by

$$C(v, a, c) = N(I(v, a; u.c)), \qquad P(y) = N(I(y, 0; u.C(u, 1, 0))),$$

where the state variable in the first expression is unused in c. Only four literal computation identities are needed:

$$C(0, a, c) = a, \quad C(Sv, a, c) = c, \quad P(1) = 1, \quad P(2) = 0, \qquad 2 = S1. \tag{3}$$

No predecessor, parity range theorem, or arithmetic induction occurs in these finite computations.

The following construction adapts the two-row witness package in the preceding witness-discharge report [2]. Its rows are exchanged: row 1 now contains the equation the rule must read automatically, and row 2 contains the formula the branch supplies. The use of the selector and parity term is inherited; the fixed-row discharge restriction is the present change.

Theorem 2.5 (Uncrossed term transport)**.** *For every fixed numerical term template $h(v)$, the new calculus derives*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash N(h(s)) = r}{\Gamma \vdash N(h(t)) = r}.$$

The retained context may contain arbitrary clauses.

Proof. Choose fresh matrix variables x, y and define the equation

$$B(x, y) = \big(C(y, 0, C(P(y), N(h(x)), x)) = C(y, 0, C(P(y), r, t))\big).$$

All displayed expressions are normalized. By (3), its relevant instances are literally

$$B(x, 0) = (0 = 0), \qquad B(x, 1) = (x = t), \qquad B(x, 2) = (N(h(x)) = r). \tag{4}$$

We first prove $D_{x,y}B$ by Intro at the witness s. For its successor branch consider the formula

$$A(v) = \big(C(v, N(h(s)), s) = C(v, r, t)\big).$$

Its zero instance is the supplied equation $N(h(s)) = r$. Every successor instance is the supplied equation $s = t$. Applying Lemma 2.4 at the target $P(Sz)$ gives

$$C(P(Sz), N(h(s)), s) = C(P(Sz), r, t),$$

which is exactly $N(B(s, Sz))$. The eigenparameter for this case proof is chosen fresh independently of z.

This successor proof does not use Intro's local hypothesis. It is therefore valid both when s is literally 0 and Intro selects progress, and when Intro selects cases. No equality congruence has been smuggled into this step: each endpoint of the case proof is one of the two given equations by literal normalization.

Apply Row to the resulting DB, opening its witness at fresh b. Equations (4) make its zero test succeed and its first row exactly $b = t$. Thus its selected target is t. In the single formula branch, Use of the opened clause at 2, with reflexivity of 0 as antecedent, gives $F = (N(h(b)) = r)$. The exported formula is $N(F[t/b])$, which is $N(h(t)) = r$ by (1). The parameter b is absent from t, r, h and the outer context, so the output scope condition holds. □

Corollary 2.6 (Equality algebra and formula transport)**.** *The new calculus derives symmetry, transitivity, term congruence, crossed one-term transport, and transport in every Eq/D formula template.*

Proof. For symmetry of $s = t$, use Theorem 2.5 with $h(v) = v$ and $r = s$; its other premise is reflexivity $s = s$, and its result is $t = s$. Applying symmetry to the conclusion of that theorem yields crossed transport $s = t,\ h(s) = r \vdash r = h(t)$. The usual instances now give transitivity and term congruence. Two crossed transports replace a parameter in both sides of an equation.

For a compound formula template, argue externally by induction on its finite syntax. Suppose it is $D_{x,y}K(v, x, y)$, and its source instance at s is proved. Open that instance at a fresh witness c. To introduce the target instance at t, choose witness c. Since this witness is a parameter rather than literal 0, Intro is in case mode. Under its hypothesis $K(t, c, 0)$, use the induction hypothesis and $t = s$ to derive $K(s, c, 0)$. Use the source clause at Sz, then the induction hypothesis and $s = t$, obtaining $K(t, c, Sz)$. Close Intro and the source elimination. Each recursive transport concerns the strictly smaller formula K.

The proof always introduces the compound target at a fresh parameter; it does not require an unproved repair of case introduction at witness 0. Its induction is external on formula syntax, not object-language induction for quantified formulas. □

2.4 Exact equivalence and its interpretation consequence

Theorem 2.7 (The entire open consequence relation is preserved)**.** *For every finite canonical clause context Γ containing $\mathcal{R}$ and every canonical formula F,*

$$\Gamma \vdash_{\mathcal{K}_{\mathrm{row}}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{K}_{=}} F.$$

Both directions are effective transformations of finite proof trees and leave the entire end sequent literally unchanged.

Proof. From the new calculus to the old, use Corollary 2.2 at every Row node. The two other instructions are unchanged.

Conversely, translate an old derivation recursively. At an old Name node, its translated branches prove $b = t$ and F under the opened clause. Use Corollary 2.6 inside that branch to obtain $N(F[t/b])$. The old rule's side condition says this result is b-free, so Lemma 2.3 discharges the witness. This construction also covers targets t that contain b; equality transport is simultaneous one-pass substitution into the fixed formula template. Use and Intro are shared.

The constructions used for this replacement were derived before this translation: ordinary elimination, then numerical cases, then the two-row term-transport package, then formula transport. Thus recursion on the old proof tree is well founded. No arbitrary numerical substitution theorem for derivations is assumed. □

Corollary 2.8 (Full first-order iPRA)**.** *The new presentation is equiconsistent and bi-interpretable with full first-order intuitionistic primitive recursive arithmetic. Both numerical interpretations use the whole domain and actual equality; both numerical composite isomorphisms have graph $x = y$.*

Proof. Compose Theorem 2.7 with the attributed canonical/raw and native/iPRA comparisons. The new comparison is the identity on numerical terms and formulas and has exactly the same ordinary reading. It therefore transfers all proofs of the inherited term composites, logical macros, and formula round trips without changing their statements. In particular, for native F and ordinary iPRA φ, the inherited statements

$$F \leftrightarrow_{\text{native}} (F^I)^J, \qquad \varphi \leftrightarrow (\varphi^J)^I$$

remain provable in the respective theories. This includes the binder round trip

$$D_{x,y}B \leftrightarrow_{\text{native}} \exists_{\text{native}} x\big(B(x,0) \rightarrow_{\text{native}} \forall_{\text{native}} y\, B(x,y)\big).$$

No restriction to standard natural numbers, proper definable cut, or quotient domain is introduced. The contradiction $0 = S0$ is unchanged, so the effective translations also give equiconsistency. □

Precisely what is not claimed. The mode in (2) is not invariant under arbitrary numerical substitution: an instance can normalize to $0 = 0$ when its unsubstituted zero row did not. Consequently, this proof does not claim that blindly substituting into a derivation preserves its primitive instruction tags. General admissibility follows through the exact equivalence instead. Nor do fewer premises imply a smaller checker without measurement: the fixed-row normalization and tests must be counted. The result establishes a stricter complete equality interface, not an absolute minimum over all possible presentations.

3 The full iPRA comparison and both interpretation composites

Here iPRA means full first-order intuitionistic primitive recursive arithmetic, not merely the quantifier-free equational calculus. Its numerical language has zero, successor, and a function symbol for every finite primitive-recursive definition generated from zero, successor, projections, composition and primitive recursion. Its nonlogical axioms are the defining equations, successor injectivity, $Sx = 0 \rightarrow \bot$, and induction for quantifier-free formulas, with arbitrary passive numerical parameters. Its logic is ordinary first-order intuitionistic logic with equality. An equivalent presentation uses induction as a contextual inference; this permits ordinary retained assumptions without adding induction for quantified formulas.

The delivered native presentation is explicitly different: Use, Intro and Name or Row are its only inference families, with the single fixed clause $\mathcal{R}$. All familiar connectives and numerical axioms are derived. The written bridge first compares it with a standard ordinary theory T_I on the scalar terms, then compares T_I with iPRA. The former has ordinary intuitionistic equality logic, both iterator computation equations, literal-equation induction, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The separation sentence is an explicit part of this ordinary realization. Without it, positive equations alone would allow a singleton model; it is not silently inferred from the notation for native contradiction.

3.1 Recovering all the logical connectives

For a normal term v and formulas A, B, extend the numerical selector $C(v, a, b) = I(v, a; u.b)$ to a formula selector $\mathrm{sel}(v, A, B)$ structurally: select corresponding numerical sides at equations, and recurse under D after freshening its variables. For different formula shapes the inherited alignment construction in Appendix B supplies common-shape representatives. Let $\mathrm{gate}(v, A)$ be the corresponding selector that is trivial at zero and equivalent to A at a successor. With $q_y A = D_{w,y} A$ for unused w, a complete set of macros is

$$\begin{aligned}
\bot &:= 0 = S0,\\
A \to B &:= q_y\, \mathrm{sel}(y, A, B),\\
A \wedge B &:= q_y\, \mathrm{gate}(y, \mathrm{sel}(P(y), A, B)),\\
\exists x\, A(x) &:= D_{x,y}\, \mathrm{gate}(y, A(x)),\\
A \vee B &:= \exists v\ \mathrm{sel}(v, A, B),\\
\forall x\, A(x) &:= A(0) \wedge q_x A(x).
\end{aligned}$$

These are formula abbreviations with explicit native expansions, not six new connectives with uncounted inference rules. The reproduced foundations prove their intuitionistic natural-deduction rules and the formula composites. In particular no excluded-middle axiom for arbitrary formulas is introduced. Decidability used later is confined to the arithmetic quantifier-free formulas for which a reflected Boolean term is constructed. The displayed universal macro eliminates by specializing $q_x A(x)$ with its $A(0)$ conjunct, and introduces from a fresh-parameter proof. It is equivalent to the zero-gated universal macro used in the inherited appendix.

3.2 Arithmetic compilation and actual equality

The scalar iterator's update sees its previous state but not an explicit recursion counter. The translation must therefore establish enough internal arithmetic to compile every primitive recursion; standard-model evaluation alone would not prove bi-interpretability. The included foundations proceed in the following dependency order. They construct a predecessor on the whole domain, derive truncated subtraction and a numerical test reflecting actual equality, compile each quantifier-free formula to an equation, derive its induction, and construct pairing with internal projection identities. An arbitrary primitive recursion then iterates a paired state consisting of its counter and current value. All these constructions use fixed finite descriptions, not a universal evaluation function.

In the reverse direction, every scalar iterator with fixed update h is a primitive-recursive definition with its two displayed computation equations. For every fixed term, the two composite translations

are proved equal to the original term internally, using equation induction and the defining equations. The equality is the original equality of the numerical domain, not an observational quotient or a restriction to standard numerals.

Theorem 3.1 (Ordinary bi-interpretability and equiconsistency)**.** *Both delivered presentations have an ordinary realization bi-interpretable with iPRA. Both interpretation domains are the entire numerical domain, interpreted equality is actual equality, and both numerical composite isomorphisms have graph $x = y$. Every formula is provably equivalent to its round-trip translation. The presentations are equiconsistent with iPRA.*

Proof. The new comparison is exact identity-language equivalence with the normal Name calculus, as proved in the preceding section. The canonical/raw comparison proved below changes a formula only by internal computation equivalence. The raw native/ordinary comparison and arithmetic compilation are given in the attributed foundations. Composing these interpretations therefore leaves the domains and equality unchanged. Compose their term identities to obtain the two numerical identity graphs. Compose their formula equivalences to obtain both logical round trips. For the consistency claim, the native contradiction $0 = S0$ and ordinary falsity are related by the explicit falsity bridge in both directions. Effective proof translations preserve this contradiction, so one theory has a contradiction if and only if the other does. □

This conclusion is stronger than agreement on true equations over the standard natural numbers. It concerns the complete first-order intuitionistic theories and their definable interpretations over their full domains. The new work does not replace the substantial inherited arithmetic bridge; it proves a smaller primitive presentation of the same consequence relation.

4 Certificate syntax and the complete trusted program

Both delivered interfaces read one JSON value from standard input. It is the two-element array `[goal, proof]`; the goal is a closed canonical formula. A run accepts precisely when it exits normally with status zero and prints exactly `ACCEPT` followed by a newline. Exceptions reject. There is no prescribed diagnostic string on rejection. The scripts are dedicated command-line checkers; they do not require a separate library wrapper or an additional input schema validator.

4.1 Finite syntax and scopes

The numerical codec uses the integer 0 for zero, positive integers for one-based bound indices, and negative integers for free numerical parameters. The successor of t is `[0,t]`, and an iterator is `[n,a,h]`, binding one variable in h only. An equation is `[s,t]`, and a paired formula is `[B]`, binding two variables in B: index 1 is y and index 2 is x. Sorts, rather than raw list length alone, distinguish a successor from an equation. The walker validates each child in its correct sort. Booleans, floats, strings, dictionaries, and malformed list arities are rejected in a used syntax position.

The retained context starts with the unconditional clause `([], [1,1])`. Every invocation of introduction or discharge adds exactly one clause and opens exactly one fresh parameter. Consequently, if the context is g, its next fresh parameter is

$$b = -|g|.$$

The currently available free parameters are $b+1, \ldots, -1$. At binder depth d, a variable leaf is in scope precisely when $b < i \leq d$. The integer 0 in this inequality is the numerical constant. This single invariant removes a separate scope counter and its update arguments. Different branches may reuse the same integer at disjoint scopes; there is no identification of their local witnesses.

The function `walk` performs structural checking, index traversal, and full computation normalization. In `syntax`, its result must be literally equal to the supplied syntax: input terms and formulas must already be canonical. The traversal checks an iterator's children before contraction, so malformed or out-of-scope data hidden in an erasable body is not accepted. The same walker implements index shifting inside `sub`; no shifting oracle or omitted term operation is used.

4.2 The three instruction shapes

Instruction	Exact JSON payload
Clause use	`["u", index, term, premise]`
Introduction	`["i", witness, branch]`
Deterministic discharge	`["e", matrix, branch_formula, major, branch]`
Explicit discharge	`["e", matrix, branch_formula, target, major, equality, branch]`

The two discharge rows belong to different checkers; each checker recognizes only its own row. The goal determines the introduction's matrix, so that matrix need not be duplicated in its payload. The discharge payload supplies the matrix of its major formula and the formula proved by the branch, because this is a goal-directed checker, not a forward proof synthesizer. Synthesis was investigated but did not yield the smaller measured program.

Clause indices must be genuine integers in range, so Python's Boolean/integer comparison aliases do not create an input loophole. An unconditional clause has no premise proof; its syntactic payload is deliberately ignored, and `null` is the standard placeholder. This is part of the protocol, not a promise to validate semantically unused JSON. Extra or missing fields in a used instruction fail its exact structural pattern.

In deterministic discharge, the major is checked before its witness clause is installed. The branch cannot leak that clause into the major proof. Opening the major yields a canonical matrix $B(b, y)$. The checker forms

$$A_0 = \mathrm{nf}(B(b,0)), \qquad (s,t) = \begin{cases} \mathrm{nf}(B(b,1)), & A_0 \equiv (0=0), \\ (0,0), & \text{otherwise.} \end{cases}$$

The first branch is safe to unpack: substitution does not change a formula's outer constructor, and A_0 being an equation implies that $B(b,1)$ is an equation. The leaf callback replaces a parameter i precisely when $i = s = b$. Thus an equality whose source is an older parameter, a constant, or a compound term does not trigger replacement. This last restriction is essential to the elementary recovery of ordinary witness elimination.

The output is normalized and compared to the already validated outer goal. That comparison also enforces absence of the discharged witness. The target may contain the witness, and inserted copies are not recursively substituted again; only a witness surviving in the normalized output is forbidden.

4.3 The invariant proved by the acceptance checker

Induct on a successful `prove` call. Its goal is a canonical formula whose free parameters lie strictly above the next-fresh cutoff. Its context consists of well-scoped optional-premise clauses. Use is exactly clause specialization; introduction opens the prescribed hypothesis at a fresh parameter; discharge invokes the explicit or deterministic inference proved in the mathematical section. Each recursive premise is checked in exactly its prescribed context. The walker's substitution and normalization lemmas give the displayed instances. Therefore successful acceptance implies derivability of the stated closed formula.

Conversely, encode a finite closed-theorem derivation starting with $\mathcal{R}$, after assigning its eigenparameters in stack order. Its scopes obey the context-length invariant. Write its ordinary clause assumptions as unconditional clauses and use the fixed initial clause for reflexivity. Induction on the derivation gives an accepted certificate. This is completeness in the usual idealization of unbounded finite storage. A concrete Python recursion or memory limit can reject a valid very large certificate; it does not authorize an invalid one.

5 Normalization, bounded checking, and the consistency claim

Each fixed finite scalar term denotes a primitive-recursive numerical function. Uniform normalization over all finite scalar descriptions is a total recursive operation, but it is not primitive recursive. A hypothetical primitive-recursive universal evaluator $V(e, n)$ for those descriptions would make $V(e, e)+1$ another member of the enumerated class, contradicting diagonalization. The normalizer is an external algorithm on syntax; the object theory contains neither a variable ranging over arbitrary descriptions nor an axiom asserting the totality of such an evaluator.

The equivalence of consequence relations and the interpretation composites in this report are mathematical metatheorems. We do not infer, merely from effective translations with normalization, that their runtime has a primitive recursive bound or that one specific arithmetization of their consistency equivalence is automatically provable in a specified weak metatheory. The ordinary realizations are bi-interpretable, so the requested ordinary equiconsistency holds.

For a primitive-recursive certificate predicate, the archive also supplies `code/checker_deterministic_fuel.py` and `code/checker_explicit_fuel.py`. Their input is `[goal, proof, fuel]`, with a nonnegative genuine integer budget. Every entry into either recursive engine, `walk` or `prove`, consumes one unit. The remaining computation at an entry is a bounded structural operation on finite data. Bounding the number of recursive entries by the supplied integer gives a primitive-recursive simulation, with a primitive-recursive bound on the sizes of intermediate data. Parsing and reading the supplied finite data are likewise bounded by their lengths.

Soundness is immediate because a successful budgeted run performs the same checks. Conversely every successful unbounded run is finite and thus has a sufficient finite budget. The budgeted and unbudgeted presentations therefore have exactly the same theorems. No uniform sufficient budget is claimed. The larger complete programs are measured separately; the budget is not silently excluded from their trusted-code counts.

6 Verification and provenance

The executable evidence is in `evidence/` and `validation/`. Proof producers and translators are untrusted: their outputs are checked by the standalone program. Translations run in both directions through the separate-rule reference calculus, and an independent forward replay checks the produced end formula. The mathematical proof of general equivalence does not rest on this finite sample.

The fixtures exercise genuine use of an equation-induction hypothesis, symmetry and transitivity from assumptions, repeated template occurrences, zero-source transport, nested iterator binders, compound-formula transport, and the zero-witness case repair. Adversarial fixtures test future and escaping eigenparameters, false equations, wrong first rows, malformed syntax, incorrect clause indices, and altered rule payloads. Both ordinary Python and Python with `-O` run the public interface; no logical check is implemented by `assert`. Budgeted runs are checked at exact and one-short budgets. The recorded results give the exact counts.

The full native-to-iPRA bridge has existing Lean source in the pinned repository [11]. The delivered reference files preserve the final statement modules `NativeBiInterpretation.lean` and `BiInterpretation.lean`; they are not a self-contained Lean build. No Lean executable was available in this run, and we did not rebuild the upstream project. The new deterministic rule, normalizing implementation, and all new comparison claims here have written proofs and executable checks; they are not claimed to be newly verified in Lean or Rocq.

The archive's attribution and hashes distinguish the pinned upstream sources, the source archives inspected, and the new files. Its manifest covers all delivered payload files except itself. Generated test output and PDF metadata can change when reproduction is run; the delivered manifest describes the original package.

A Computation normalization and the raw comparison

This appendix states the inherited normalization argument explicitly, since moving computation into syntax must count both its algorithm and its metatheoretic justification [6, 2]. Reduction is the contextual closure, including inside update bodies, of

$$I(0, a; u.h) \to a, \qquad I(Sn, a; u.h) \to h[I(n, a; u.h)/u].$$

Capture-avoiding substitution preserves reduction. For a strongly normalizing finite term t, let $\nu(t)$ be the largest length of a reduction sequence from t, and let $|t|$ be its syntax size. The maximum exists because the finitely branching reduction tree is well founded.

Lemma A.1 (Iterator closure)**.** *If n, a are strongly normalizing and $h[v/u]$ is strongly normalizing for every strongly normalizing v, then $I(n, a; u.h)$ is strongly normalizing.*

Proof. The body is strongly normalizing by taking $v = u$. Induct lexicographically on $(\nu(n), |n|, \nu(a) + \nu(h))$. A contraction within n lowers the first coordinate; one in a or h lowers the third. Substitution stability preserves the universal hypothesis on a reduced body. The zero root reduct is a. At a successor root $n = Sp$, the counter p has the same reduction height and smaller size, so the induction hypothesis makes $I(p, a; u.h)$ strongly normalizing. The premise about h makes its substitution into the successor reduct strongly normalizing. Thus every immediate reduct is strongly normalizing. □

Theorem A.2 (Normalization and substitution coherence)**.** *Every finite raw term is strongly normalizing and has a unique normal form, up to bound-variable renaming. With de Bruijn indices*

uniqueness is literal. For every substitution σ,

$$\mathrm{nf}(t[\sigma]) = \mathrm{nf}\big(\mathrm{nf}(t)[x \mapsto \mathrm{nf}(\sigma x)]\big).$$

The same property holds for formulas.

Proof. Prove by structural induction that replacing a fixed term's free variables by any strongly normalizing terms yields a strongly normalizing term. At an iterator, the structural hypotheses for its counter, seed, and body are exactly the preceding lemma's premises. Identity substitution gives strong normalization. Local confluence follows because the two root patterns do not overlap, disjoint contractions commute, and a contraction within a root redex has finitely many residual copies after that root contraction. Contract every retained copy; erased copies require no step. Substitution stability gives a common reduct. Well-founded induction on reduction height now gives confluence. Apply confluence to reducing t before substitution and reducing the supplied arguments before substitution to obtain the displayed identity. Formula constructors themselves do not contract, so the formula statement follows structurally. □

The delivered normalizer first traverses the children. At a successor counter it normalizes the predecessor iterate and inserts that result into the update by capture-avoiding substitution. Its contractions are precisely the stated reductions, interspersed with finite structural traversals. The theorem identifies its result with the unique normal form and justifies termination in unbounded finite memory. Section 5 separates this total-recursive algorithm from the optional bounded certificate predicate.

A.1 A computation inside an update is provably sound internally

In the raw Choice-DI calculus of Appendix B, equation induction and ordinary equality reasoning are derived. If u is fresh for Γ and $\Gamma \vdash h(u) = k(u)$, then

$$\Gamma \vdash I(n, a; u.h) = I(n, a; u.k).$$

For this update-extensionality lemma, induct on the equation comparing the two iterates at a fresh counter z. At zero both are a. In the successor step, instantiate the supplied pointwise equality at the first iterate, then use the induction equation inside k to identify the two updates. Only literal-equation induction is used. This lemma matters because a free-parameter transport rule alone would not justify capture of that parameter by an iterator binder.

Every contextual computation step is consequently an internal equality: the root equations are axioms, usual congruence covers successor, counter and seed positions, and the lemma covers update bodies. Thus every fixed term satisfies $t = \mathrm{nf}(t)$ internally, and formula transport gives $A \dashv\vdash \mathrm{nf}(A)$.

A.2 The zero-witness case repair

Full normalization removes nonliteral closed aliases for zero. The following inherited compound-envelope argument repairs a case-introduction proof whose witness normalizes to zero [6, 2]. It is a derived construction, not an extra primitive rule.

For an equation $E = (s = t)$ define

$$G_v E = (C(v, 0, s) = C(v, 0, t)), \qquad L(E) = q_v G_v E.$$

The endpoints are literally $G_0E = (0 = 0)$ and $G_{Sw}E = E$. The closure rules of Lemma 2.4 give $E \vdash L(E)$ and $L(E) \vdash E$, using reflexivity and specialization at 1 for the second.

Suppose a supplied derivation proves $\Gamma, E(0,0) \vdash E(0,Sz)$. Put $A(x,y) = L(E(x,y))$. Under $A(0,0)$, unwrap the envelope, insert that derivation by ordinary assumption cut, and wrap its conclusion. Intro at witness zero now uses cases, because A is compound, and produces $D_{x,y}A(x,y)$. Open it at fresh c. To introduce $D_{x,y}E(x,y)$ at witness c, assume $E(c,0)$, wrap it as $A(c,0)$, apply the opened clause at Sz, and unwrap $A(c,Sz)$. Since c is not literal zero, this introduction again uses cases. Closing the ordinary elimination proves the required $D_{x,y}E(x,y)$. Thus unrestricted case introduction is available even at a zero witness.

A.3 Exact raw/canonical comparison

Normalize the terms of a raw Choice-DI derivation into an auxiliary canonical calculus with separate case and equation-progress introductions. Computation equations become reflexivity and normalized substitution coherence supplies the remaining instances. The preceding repair supplies arbitrary case introduction in the delivered calculus.

For arbitrary witnessed equation progress at a term t, first introduce $D_{w,y}E(t,y)$ with unused w at witness zero. This uses precisely the progress hypothesis supplied. Open its conditional closure, use it under $E(t,0)$ to prove the successor instance, and then use derived case introduction to produce $D_{x,y}E(x,y)$ at t. Equation transport is provided by the preceding new-rule theorem; replacing a fixed reflexivity rule by instances of $\mathcal{R}$ changes no consequence.

Conversely, the raw calculus simulates the delivered rules using ordinary paired elimination, its derived equality transport, and the appropriate case or equation-progress introduction. Internal normalization equivalences convert each canonical instance to its raw counterpart and back. The canonical-to-raw-to-canonical composite is literal identity. The other composite is the internally proved $A \dashv\vdash \mathrm{nf}(A)$. This proves the raw/canonical comparison without assuming that a nonzero raw witness remains nonzero after normalization, or that every rule tag survives arbitrary numerical substitution.

Attribution and role of the following foundations

The next five appendices reproduce the logical and arithmetic reference development from the pinned ProveIt sources [7, 8, 9, 10], as reproduced in the preceding witness-discharge report [2]. They are licensed under the included MIT No Attribution license. They establish the complete logical interface, predecessor, reflected equality, quantifier-free induction, pairing, primitive-recursion compilation and both interpretation composites. They introduce no additional primitive rule into either delivered checker. Their historical labels "old" and "new" refer to the inherited comparisons, not to Row in the present report.

B Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

B.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F} \quad \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)} \quad \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

B.2 Exact equivalence and the unused-witness construction

Lemma B.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem B.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi : \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition B.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \le 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

B.3 Induction and numerical cases separate cleanly

Lemma B.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma B.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

B.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \qquad \text{(Gate and padding)}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and

$p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

B.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)). \qquad \text{(Paired composite)}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem B.2 preserves every native sequent.

B.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

C Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two

equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section B recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

C.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.

2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.

3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.

4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem C.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

D Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [8] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

D.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{5}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{6}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

D.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{7}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{8}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{9}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (9) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{10}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (9); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

D.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{11}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (10), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{12}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{13}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

D.4 Saturated iteration gives predecessor

We now need only the three equations (13). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \end{aligned} \tag{14}$$

$$B_{Sm}(0) = C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \tag{15}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \tag{16}$$

The base is (15). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \tag{17}$$

The first uses the comparator recurrence. The second uses (16) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{18}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{19}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (18) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

D.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

E From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [9]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix D, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

E.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{20}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{21}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{22}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{23}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{24}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (24) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

E.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{25}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{26}$$

$$K(0) = 1, \qquad K(Sx) = x. \tag{27}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (20). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (26); symmetry and (23) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{28}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{29}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{30}$$

To prove the range assertion, split (22). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (21) also give, for all u, v,

$$C(u, v, v) = v. \tag{31}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{32}$$

If $u = 1 = S0$, (27) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

E.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{33}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{34}$$

Using these equations and (24), we now derive

$$0 + y = y, \tag{35}$$
$$(Sx) + y = S(x + y), \tag{36}$$
$$d(Sx, Sy) = d(x, y), \tag{37}$$
$$d(Sa, a) = 1. \tag{38}$$

Each induction below has a single equation as its induction formula.

For (35), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (36), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (36) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{39}$$

Indeed, both sides are $S(y + r)$.

For (37), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (38). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (37) and the induction hypothesis.

E.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)\big) = x. \tag{40}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (35) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (31).

For the step, assume (40) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (40) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (39) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (40). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{41}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (40). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (38) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{42}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{43}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{44}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (29), which is equivalent to $d(Sa, b) = 1$ by (32), and then to $a = b$ by (41). The range assertion is (28) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (43) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{45}$$

is a theorem, not an extra hypothesis.

E.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{46}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{47}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{48}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{49}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{50}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (43)–(44). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (50) for every quantifier-free formula.

E.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (50), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation

induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

F Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [10]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

F.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v + 1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n + 1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n) + 1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \leq k^2 + a < (k+1)^2$ since $a \leq k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

F.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

F.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section B give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

G Complete standalone acceptance programs

These listings contain all code used for unbounded acceptance. Only Python's standard JSON module is imported. The explicitly bounded variants are separately supplied and measured.

G.1 Explicit discharge: the smaller checker

```
import json

def need(condition):
    if not condition:
        raise ValueError('invalid certificate')

def walk(a, f, d=0, form=None):
    if type(a) is int:
        need(not form)
        return f(a, d)
    need(type(a) is list)
    size = len(a)
    need(form is None or (size in ((1, 2) if form else (2, 3)) and (form or size == 3 or a[0] == 0)))
    a = [walk(b, f, d + (2 if size == 1 else int(size == 3 and i == 2)), size == 1) for i, b in enumerate(a)
     ]
    if size == 3:
        c, b, h = a
        if c == 0:
            return b
        if type(c) is list and len(c) == 2:
            return sub(h, walk([c[1], b, h], lambda i, d: i))
    return a

def syntax(a, fresh, form=False):

    def variable(i, d):
        need(fresh < i <= d)
        return i
    need(walk(a, variable, form=form) == a)

def sub(a, t, depth=0):
    return walk(a, lambda i, d: walk(t, lambda j, e: j + d if j > e else j) if i == d + 1 else i - (i > d +
     1), depth)

def prove(a, p, g):
    fresh = -len(g)
    syntax(a, fresh, True)
    match p:
        case ['u', j, t, premise]:
            need(type(j) is int and 0 <= j < len(g))
            syntax(t, fresh)
            base, body = g[j]
            need(a == sub(body, t))
            if base:
                prove(base, premise, g)
        case ['e', body, fml, t, major, eq, branch]:
            prove([body], major, g)
            body = sub(body, fresh, 1)
            local = g + [(sub(body, 0), body)]
            prove([fresh, t], eq, local)
            prove(fml, branch, local)
            need(walk(fml, lambda i, d: t if i == fresh else i) == a)
        case ['i', t, branch]:
            need(len(a) == 1)
            syntax(t, fresh)
            body = sub(a[0], t, 1)
            hyp = sub(body, fresh if len(body) == 2 and t == 0 else 0)
```

```
            prove(sub(body, [0, fresh]), branch, g + [([], hyp)])
        case _:
            raise ValueError('invalid proof')

goal, proof = json.load(open(0))
prove(goal, proof, [([], [1, 1])])
print('ACCEPT')
```

G.2 Deterministic discharge: the new rule

```
import json

def need(condition):
    if not condition:
        raise ValueError('invalid certificate')

def walk(a, f, d=0, form=None):
    if type(a) is int:
        need(not form)
        return f(a, d)
    need(type(a) is list)
    size = len(a)
    need(form is None or (size in ((1, 2) if form else (2, 3)) and (form or size == 3 or a[0] == 0)))
    a = [walk(b, f, d + (2 if size == 1 else int(size == 3 and i == 2)), size == 1) for i, b in enumerate(a)
     ]
    if size == 3:
        c, b, h = a
        if c == 0:
            return b
        if type(c) is list and len(c) == 2:
            return sub(h, walk([c[1], b, h], lambda i, d: i))
    return a

def syntax(a, fresh, form=False):

    def variable(i, d):
        need(fresh < i <= d)
        return i
    need(walk(a, variable, form=form) == a)

def sub(a, t, depth=0):
    return walk(a, lambda i, d: walk(t, lambda j, e: j + d if j > e else j) if i == d + 1 else i - (i > d +
     1), depth)

def prove(a, p, g):
    fresh = -len(g)
    syntax(a, fresh, True)
    match p:
        case ['u', j, t, premise]:
            need(type(j) is int and 0 <= j < len(g))
            syntax(t, fresh)
            base, body = g[j]
            need(a == sub(body, t))
            if base:
                prove(base, premise, g)
        case ['e', body, fml, major, branch]:
            prove([body], major, g)
            body = sub(body, fresh, 1)
            base = sub(body, 0)
            local = g + [(base, body)]
            s, t = sub(body, [0, 0]) if base == [0, 0] else (0, 0)
            prove(fml, branch, local)
            need(walk(fml, lambda i, d: t if i == s == fresh else i) == a)
        case ['i', t, branch]:
```

```
            need(len(a) == 1)
            syntax(t, fresh)
            body = sub(a[0], t, 1)
            hyp = sub(body, fresh if len(body) == 2 and t == 0 else 0)
            prove(sub(body, [0, fresh]), branch, g + [([], hyp)])
        case _:
            raise ValueError('invalid proof')

goal, proof = json.load(open(0))
prove(goal, proof, [([], [1, 1])])
print('ACCEPT')
```

H Reproduction and file map

The mathematical report is the single standalone source `report.tex`; it embeds all mathematical appendices and both unbounded checkers. The editable section sources and deterministic assembler are also included. Python verification uses the standard library only. Rebuilding the PDF requires the usual LaTeX packages named in the preamble.

```
python3 verify_manifest.py
python3 reproduce.py
python3 code/checker_deterministic.py \
  < validation/examples/successor_congruence.json
python3 build_pdf.py
```

The directory `code/` contains four complete standalone checkers: explicit or deterministic discharge, each with or without an explicit computation budget. The ordinary interfaces use `[goal,proof]`; the budgeted interfaces use `[goal,proof,fuel]`. The final proof examples in `validation/examples/` use deterministic discharge. The archived old-Name examples in `reference/old_name_examples/` have an object wrapper; replace that wrapper by `[goal,proof]` to run them directly with the delivered explicit checker. The verification script performs that conversion automatically.

The `validation/` directory contains proof constructors, translations, a separate-rule replay kernel, an independent named-variable normalizer and replayer, theorem fixtures and adversarial fixtures. None is imported by an acceptance checker. The `evidence/` directory records exact code metrics, normalized copies and maximum-compression gzip files, test results and PDF checks. The `reference/` directory preserves attributed frozen sources and the exact final Lean theorem statements inspected. Those two Lean files are references rather than a complete buildable Lean project.

References

[1] V. Reshetnikov, ed., *Concise presentations of intuitionistic primitive recursive arithmetic*, ProveIt. Frozen revision `ffa94f76bc01d1c77168f035caed949179859115`. Pinned repository directory.

[2] *iPRA with equality at witness discharge*, `docs/incoming/ipra_witness_discharge_2026-11-11.zip`, in [1]. Source of the explicit Name calculus, full-normal interface, parity packet and the immediate comparison checker. Frozen archive.

[3] *iPRA with uniform binding*, `docs/incoming/ipra_uniform_binding_2026-11-11.zip`, in [1]. Counter-aware scalar checker with a shared traversal and context-derived scope; the frozen implementation is included.

[4] *Four-rule iPRA with reusable clauses*, `docs/incoming/ipra_clause_contexts_2026-11-11.zip`, in [1]. The selected packet checker is reported as 2,282 bytes, 666 tokens, and 770 AST nodes.

[5] *Canonical recursors* and *Checked refinement*, `Research/canonical_recursors/compact/code/checker_compact_zero.py` and `Research/checked_refinement/code/checker_native.py`, in [1]. Distinct 516-byte and 788-byte comparison interfaces.

[6] *Full normalization*, `Research/full_normalization`, in [1]. Strong normalization, confluence, normalized substitution, and the compound-envelope zero-witness repair; also reproduced in [2].

[7] *Certified zero*, `Research/certified_zero/reference`, in [1]. Sources `inherited_choice_basis.tex` and `inherited_ordinary_bridge.tex`; reproduced here with attribution.

[8] *Scalar predecessor bootstrap*, `Research/certified_zero/reference/scalar_predecessor.tex`, in [1]. Factor map, comparator, and saturated-iteration construction.

[9] *Equality reflection*, `Research/certified_zero/reference/equality_reflection.tex`, in [1]. Internal reflection of actual equality and compilation of quantifier-free formulas.

[10] *Pairing and primitive recursion*, `Research/certified_zero/reference/primitive_recursion.tex`, in [1]. Scalar pairing, primitive-recursive compiler and internal term-composite identities.

[11] *ConciseIPRA Lean formalization*, especially `Lean/ConciseIPRA/NativeBiInterpretation.lean` and `Lean/ConciseIPRA/BiInterpretation.lean`, in [1]. Native-to-iPRA statements; Scalar-to-iPRA statements. No upstream build is claimed for the present run.